ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ (НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)»

Компьютерные науки и прикладная математика

УДК 004.89

СОГЛАСОВАНО

Научный руководитель,
Кандидат физико-математических
наук, доцент

_____ А. М. Романенков
«_____» _____ 2026 г.

УТВЕРЖДАЮ

Директор инстстута
Компьютерные науки и
прикладная математика, кандидат
физико-математических наук, доцент

_____ С. С. Крылов
«_____» _____ 2026 г.

Выпускная квалификационная работа
(академическая)

на тему: **Проектирование и реализация высокопроизводительных криптографических примитивов Curve25519/Ed25519 для .NET**

по направлению подготовки 02.03.02 «Фундаментальная информатика и информационные технологии»

ВЫПОЛНИЛ

студент группы М8О-411Б-22
образовательной программы
02.03.02 «Фундаментальная
информатика и
информационные технологии»

_____ М. С. Губарев
«_____» _____ 2026 г.

Москва 2026

РЕФЕРАТ

Итоговая аттестационная работа состоит из 38 страниц, 4 глав, 9 рисунков, 6 листингов, 1 приложения. Использовано 10 источников.

CURVE25519; ED25519; .NET; C#; SIMD; AVX2; КРИПТОГРАФИЯ; ZERO ALLOCATION

Объект разработки: процесс программной реализации алгоритмов эллиптической криптографии для экосистемы .NET.

Цель работы: проектирование и реализация высокопроизводительных, кроссплатформенных криптографических примитивов Curve25519 и Ed25519 на управляемом языке C#14 с применением техник аппаратного ускорения и оптимизации работы с памятью.

Методы и методология: математическое моделирование арифметики конечных полей с использованием избыточного представления Radix-51; применение полных законов сложения (кривые Эдвардса) для обеспечения защиты от атак по времени; векторизация вычислений с применением SIMD-инструкций (AVX2); использование паттернов Zero Allocation (Span<T>, ref struct) для исключения накладных расходов на сборку мусора; бенчмаркинг.

Результаты работы и их новизна: разработана полностью нативная для платформы .NET библиотека, реализующая обмен ключами X25519 [1] и электронную подпись Ed25519 [2] за строго константное время. В отличие от широко распространенных библиотек (например, Bouncy Castle), в данной работе реализован подход пакетной векторизации на базе интринсиков AVX2, что позволило достичь превосходства в производительности от 30% присохранении полной управляемости кода.

Область применения: результаты работы ориентированы на использование в высоконагруженных серверных системах и облачных микросервисах на платформе .NET, где критически важна пропускная способность криптографического ядра. Разработанная библиотека может быть интегрирована в реализации протокола TLS 1.3 для ускорения процесса установления защищенных соединений (Handshake), в системы обработки цифровых подписей Ed25519 в реальном времени, а также в IoT-шлюзы и Edge-вычисления, аккумулирующие потоки данных от множества устройств. Отсутствие зависимости от нативного кода (P/Invoke) делает решение хорошим выбором для использования в изолированных контейнеризированных средах (Docker, Kubernetes) и других схожих подходах.

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ **5**

ВВЕДЕНИЕ **7**

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И МАТЕМАТИЧЕСКИЙ АППАРАТ КРИПТОГРАФИЧЕСКИХ ПРИМИТИВОВ **10**

1.1 Арифметика конечного поля $GF(2^{255} - 19)$ 10

1.2 Выбор представления чисел: Radix-51 10

1.3 Операции деления и извлечения квадратного корня в $\mathbb{F}_p$ 11

1.4 Математика эллиптических кривых и проективные координаты 12

1.4.1 Сложение точек 13

1.4.2 Поиск обратного элемента 13

1.4.3 Нахождение квадратного корня 14

1.4.4 Применение проективных координат в вычислениях 14

1.4.5 Лестница Монтгомери и дифференциальное сложение 15

1.4.6 Полный закон сложения кривой Эдвардса 16

1.5 Векторизация и SIMD-ускорение (AVX2) 17

2 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ И ПРОГРАММНЫХ РЕШЕНИЙ ВЫСОКПРОИЗВОДИТЕЛЬНОЙ КРИПТОГРАФИЧЕСКОЙ БИБЛИОТЕКИ **18**

2.1 Требования к программному комплексу 18

2.2 Многоуровневая архитектура библиотеки 19

2.3 Проектирование структур данных вычислительного ядра 19

2.3.1 Архитектура векторизованного представления Radix-51: AoS против SoA 20

2.3.2 Паттерн проектирования для константного времени (Constant-Time SIMD) 21

2.4 Взаимодействие с данными: транспонирование AoS и SoA 21

2.5 Модель управления памятью и паттерн Zero Allocation 21

2.5.1 Использование структур по ссылке (ref struct) 22

3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И АНАЛИЗ ПРОИЗВОДИТЕЛЬНОСТИ КРИПТОГРАФИЧЕСКИХ ПРИМИТИВОВ **23**

3.1 Выбор инструментария и среды разработки 23

3.2 Реализация вычислительного ядра (Field Arithmetic) 23

3.2.1 Векторизация и пакетная обработка (AVX2) 23

3.3 Реализация защиты от атак по сторонним каналам 25

3.4 Бенчмаркинг и сравнительный анализ производительности 26

3.5 Условия проведения тестирования 26
3.6 Результаты тестирования 27
3.7 Анализ полученных результатов 30

4 АНАЛИЗ РЕЗУЛЬТАТОВ И ПЕРСПЕКТИВЫ РАЗВИТИЯ 32
4.1 Оценка эффективности разработанной криптографической библиотеки . 32
4.1.1 Сопоставление полученных результатов с поставленными задачами 32
4.1.2 Анализ преимуществ и недостатков реализованного решения . . . 33
4.2 Обсуждение результатов бенчмаркинга и профилирования 34
4.3 Определение направлений для дальнейшего развития продукта 34

ЗАКЛЮЧЕНИЕ 36

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 38

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

.NET (.NET 10) – кроссплатформенная среда выполнения и фреймворк разработки от Microsoft 10 версии, поддерживающая множество языков программирования и предоставляющая обширную стандартную библиотеку

AVX2 (Advanced Vector Extensions 2) – набор SIMD-инструкций x86, позволяющий выполнять математические операции над 256-битными регистрами

Bouncy Castle – популярная кроссплатформенная криптографическая библиотека, дефакто являющаяся стандартом для многих управляемых (managed) сред

Constant-time (Константное время выполнения) – свойство программной реализации криптографического алгоритма, при котором время его работы не зависит от значений секретных ключей или обрабатываемых данных

Curve25519 / X25519 – эллиптическая кривая в форме Монтгомери, используемая преимущественно для протоколов алгоритма Диффи-Хеллмана (ECDH)

Ed25519 / EdDSA – схема электронной цифровой подписи на основе искривленных эллиптических кривых Эдвардса, использующая хеш-функцию SHA-512

Intrinsics (Интринсики) – специальные методы в платформе .NET (System.Runtime.Intrinsics), транслируемые JIT-компилятором напрямую в специфичные машинные инструкции процессора [3]

Radix-51 – представление 255-битного числа в виде массива из пяти 64-битных элементов (лимбов), в каждом из которых значащими являются только младшие 51 бит

SIMD (Single Instruction, Multiple Data) – архитектурный принцип построения процессоров, позволяющий выполнять одну машинную инструкцию над набором данных одновременно

Zero Allocation (Нулевые аллокации) – подход к программированию в .NET, при котором в ходе выполнения критического пути кода не выделяется память в управляемой куче (heap), что исключает паузы на работу сборщика мусора

Атака по сторонним каналам (Side-Channel Attack) – класс криптографических атак, использующих информацию о физическом процессе выполнения алгоритма (например, время выполнения, энергопотребление), а не слабости самого алгоритма

Векторизация (Vectorization) – процесс преобразования алгоритма, оперирующего одиночными значениями (скалярами), в алгоритм, одновременно обрабатывающий массив данных с помощью SIMD-инструкций

Конечные поля (Finite Fields) – алгебраическая структура, состоящая из конечного множества элементов, для которых определены операции сложения, вычитания, умножения и деления

Пакетная обработка (Batching) – метод группировки нескольких независимых криптографических операций для их параллельного выполнения на одном процессорном ядре

Проективные координаты (Projective Coordinates) – система координат, позволяющая представлять точки эллиптической кривой в виде дробей, что исключает необходимость выполнения ресурсоемкой операции деления на каждом шаге сложения

Сборщик мусора (GC, Garbage Collector) – компонент платформы .NET, обеспечивающий автоматическое управление памятью и освобождающий память, занятую неиспользуемыми объектами

ВВЕДЕНИЕ

Актуальность темы исследования

С переходом на стандарт TLS 1.3 алгоритмы на кривых Curve25519 и Ed25519 стали основной современной криптографии. Однако в экосистеме .NET разработчики до сих пор сталкиваются с выбором: использовать быстрые нативные библиотеки через P/Invoke, жертвуя кроссплатформенностью и простотой развёртки, либо выбирать полностью управляемые реализации (managed), которые часто работают значительно медленнее из-за отсутствия низкоуровневых оптимизаций. В условиях высокой нагрузки криптографические вычисления часто становятся «узким местом» системы. Это делает задачу создания высокопроизводительной библиотеки на чистом C, способной эффективно использовать SIMD-инструкции процессора, актуальной для современной разработки.

Степень разработанности проблемы

На сегодняшний день существует ряд реализаций эллиптической криптографии для экосистемы .NET:

1) Обертки над нативными библиотеками (например, Libsodium [4]). Обеспечивают высокую скорость, однако требуют компиляции под каждую целевую операционную систему и архитектуру (x64, ARM), а вызов неуправляемого кода (P/Invoke overhead) нивелирует преимущество скорости на коротких операциях.

2) Встроенные средства ОС (`System.Security.Cryptography`). Ограничены платформой Windows (CNG) или Linux (OpenSSL) и не всегда поддерживают современные стандарты кривых в одинаковом объеме.

3) Управляемые (managed) реализации, такие как индустриальный стандарт Bouncy Castle [5]. Гарантируют 100% переносимость, но исторически спроектированы под 32-битные архитектуры (использование представления Radix-25.5), активно аллоцируют объекты в куче и не применяют векторные инструкции (SIMD), что критически ограничивает их производительность.

В итоге на платформе .NET не хватает инструментов, которые сочетали бы удобство чистого C с производительностью нативного кода. Использование SIMD-инструкций и глубокая оптимизация алгоритмов позволяют создать решение, которое не уступает библиотекам на языке C по скорости, но при этом остается безопасным и простым в развертывании.

Проблема исследования

Основная сложность в том, что при использовании стандартных средств .NET трудно добиться той же производительности, которую дают нативные библиотеки. Разработчик часто оказывается перед выбором: либо писать понятный и переносимый управляемый код, либо выжимать максимум скорости, используя специфические инструкции процессора (AVX2). Для криптографических систем, работающих под большой нагрузкой, этот компромисс критичен, так как обычные методы вычислений на

C не позволяют полностью задействовать мощность современного «железа». Поэтому возникает необходимость в разработке библиотеки, которая бы использовала все возможности платформы .NET для повышения производительности, не жертвуя при этом удобством и безопасностью кода.

Цель работы

Целью выпускной квалификационной работы является проектирование и реализация высокопроизводительной кроссплатформенной библиотеки криптографических примитивов X25519 и Ed25519 для платформы .NET, обеспечивающей выполнение вычислений за константное время с использованием технологий аппаратного ускорения SIMD и паттернов Zero Allocation.

Задачи исследования

Для достижения поставленной цели в ходе работы были решены следующие задачи:

1) Изучить математические особенности кривых Монтгомери и Эдвардса, а также особенности вычислений в поле $GF(2^{255} - 19)$.

2) Спроектировать вычислительное на базе представления Radix-51, оптимизированное под 64-битные регистры и минимизацию операций переноса.

3) Разработать архитектуру библиотеки, исключающую аллокации в куче (Zero Allocation) за счет использования `Span<byte>`, `stackalloc` и типов `ref struct`.

4) Реализовать алгоритмы пакетной обработки (batching) с применением векторных инструкций AVX2 через механизм `System.Runtime.Intrinsics`.

5) Сравнить производительность и потребление памяти разработанного решения с библиотекой Bouncy Castle с помощью инструментов микро-бенчмаркинга.

Объект и предмет исследования

Объектом исследования выступают алгоритмы эллиптической криптографии Curve25519 и Ed25519.

Предметом исследования являются программные и аппаратные методы эффективной реализации арифметики конечных полей в среде исполнения .NET.

Методы исследования

В работе применялись методы математического моделирования (модульная арифметика, проективная геометрия эллиптических кривых), методы структурного и объектно-ориентированного проектирования программного обеспечения, а также эмпирические методы: профилирование производительности, юнит-тестирование по тестовым векторам (Test Vectors) и бенчмаркинг.

Научная новизна и практическая значимость

Научная новизна работы заключается в адаптации подходов аппаратных реализаций (FPGA) к управляемой среде выполнения. Использован подход пакетной векторизации (AVX2 Batching) для кривой Curve25519 полностью на языке C# без использования небезопасного стороннего кода (unsafe C/C++ bindings).

Практическая значимость: результатом работы является высокопроизводительная программная библиотека, реализующая криптографические примитивы X25519 и Ed25519. Библиотека соответствует стандартам RFC 7748 и RFC 8032, не имеет внешних зависимостей и готова к интеграции в высоконагруженные .NET-проекты. Исходный код реализации и данные для доступа к репозиторию приведены в А.

Апробация результатов

Основные положения и результаты работы докладывались на LII Международной молодёжной научной конференции «Гагаринские чтения» (Москва, МАИ, 2026 г.). По материалам выступления опубликованы тезисы доклада в сборнике научных трудов конференции [6].

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И МАТЕМАТИЧЕСКИЙ АППАРАТ КРИПТОГРАФИЧЕСКИХ ПРИМИТИВОВ

Реализация Curve25519 и Ed25519 требует не простого следования стандартам, а выбора конкретных математических представлений, пригодных для высокопроизводительных вычислений. Ниже приведено теоретическое обоснование работы с конечными полями и эллиптическими кривыми, а также описаны подходы к аппаратному ускорению, которые легли в основу практической части данной работы.

1.1 Арифметика конечного поля $GF(2^{255} - 19)$

В основе алгоритмов Curve25519 и Ed25519 лежат вычисления в конечном простом поле $\mathbb{F}_p$, где простое число задано как $p = 2^{255} - 19$. Выбор данного числа (псевдо-Мерсенновское простое) Д. Бернштейном [7] не случаен: оно оптимизировано для эффективной программной реализации операций редукции по модулю.

1.2 Выбор представления чисел: Radix-51

Стандартные 64-битные типы данных недостаточны для хранения 255-битных чисел. Традиционный подход, используемый во многих библиотеках (например, Bouncy Castle), заключается в разбиении числа на 10 элементов (лимбов) по 25.5 бит (Radix-25.5). Однако данный подход требует постоянного приведения типов при работе на 64-битных архитектурах.

В рамках данной работы выбрано избыточное представление Radix-51. Число $X \in \mathbb{F}_p$ представляется в виде полинома:

$$X = \sum_{i=0}^{4} x_i \cdot 2^{51i} \tag{1}$$

где x_i — 64-битные беззнаковые целые числа (ulong).

Каждый элемент хранит 51 бит полезной информации, оставляя 13 бит запаса в 64-битном регистре. Этот запас позволяет выполнять многократные сложения и умножения без риска переполнения и без немедленного выполнения дорогостоящей операции обработки переносов (carry propagation). Данный подход, известный как «отложенный перенос», широко применяется при проектировании аппаратных криптоускорителей (FPGA) [8] и был адаптирован для программной среды .NET.

Свойство $2^{255} \equiv 19 \pmod p$ позволяет заменять умножение старших разрядов (сдвиги на 2^{255}) на быстрое умножение на константу 19. Как показывают выкладки, перемножение двух чисел в Radix-51 требует всего 25 скалярных умножений, а возведение в квадрат — 15 умножений, что значительно снижает вычислительную сложность по

сравнению с Radix-25.5.

Математически умножение двух элементов поля F и G в представлении Radix-51 сводится к перемножению полиномов степени 4:

$$F \cdot G = \left(\sum_{i=0}^{4} L_i x^i \right) \cdot \left(\sum_{j=0}^{4} K_j x^j \right) \tag{2}$$

где $x = 2^{51}$. При раскрытии скобок образуются члены вплоть до x^8. Базовое свойство поля $2^{255} \equiv 19 \pmod p$ позволяет выполнить подстановку $x^5 = 19$. Таким образом, степени выше четвертой редуцируются: $x^5 = 19$, $x^6 = 19x$, $x^7 = 19x^2$, и так далее.

Группировка членов при одинаковых степенях x дает итоговые формулы для вычисления промежуточного результата (до финальной нормализации переносов):

$$\begin{aligned}
C_0 &= L_0K_0 + 19(L_1K_4 + L_2K_3 + L_3K_2 + L_4K_1) \\
C_1 &= L_0K_1 + L_1K_0 + 19(L_2K_4 + L_3K_3 + L_4K_2) \\
C_2 &= L_0K_2 + L_1K_1 + L_2K_0 + 19(L_3K_4 + L_4K_3) \\
C_3 &= L_0K_3 + L_1K_2 + L_2K_1 + L_3K_0 + 19(L_4K_4) \\
C_4 &= L_0K_4 + L_1K_3 + L_2K_2 + L_3K_1 + L_4K_0
\end{aligned} \tag{3}$$

Данная система требует выполнения ровно 25 операций умножения.

Для операции возведения в квадрат ($F \cdot F$) вычисления дополнительно оптимизируются. Поскольку перекрестные члены совпадают ($L_iK_j = L_jK_i$), они удваиваются ($2L_iL_j$), что позволяет сократить общее количество умножений с 25 до 15, значительно ускоряя алгоритм.

1.3 Операции деления и извлечения квадратного корня в $\mathbb{F}_p$

Извлечение квадратного корня. Операция необходима в протоколе Ed25519 (RFC 8032) для восстановления координаты x из публичного ключа, где передается только y и знаковый бит. Требуется решить уравнение $X^2 \equiv a \pmod p$.

Поскольку $p = 2^{255} - 19$, рассмотрим остаток от деления p на 8. Так как $255 = 3 \cdot 85$, число 2^{255} делится на 8 нацело. Следовательно, $-19 \pmod 8 \equiv -19 + 24 = 5 \pmod 8$. Таким образом, $p \equiv 5 \pmod 8$. Для таких полей существует алгоритм извлечения корня из 4 шагов:

1. Согласно критерию Эйлера, корень из a существует (является квадратичным вычетом), если $a^{(p-1)/2} \equiv 1 \pmod p$. 2. Вычисляется кандидат на корень: $X_0 \equiv a^{(p+3)/8} \pmod p$. Возведем кандидата в квадрат:

$$X_0^2 \equiv a^{\frac{p+3}{4}} \equiv a^{1+\frac{p-1}{4}} \equiv a \cdot a^{\frac{p-1}{4}} \pmod p \tag{4}$$

3. Введем обозначение $\beta \equiv a^{(p-1)/4} \pmod p$. Заметим, что $\beta^2 \equiv a^{(p-1)/2} \equiv 1 \pmod p$ (из критерия Эйлера). Следовательно, β может принимать значения только 1 или -1. 4.

Окончательный выбор корня основывается на значении параметра β. Математически это выражается следующей зависимостью:

$$X = \begin{cases} X_0, & \text{если } \beta = 1 \pmod p \\ X_0 \cdot 2^{(p-1)/4} \pmod p, & \text{если } \beta = -1 \pmod p \end{cases} \tag{5}$$

Обоснование корректности второго случая вытекает из свойств квадратичных невычетов. Для простых чисел вида $p = 8k + 5$ число 2 всегда является квадратичным невычетом по критерию Эйлера: $2^{(p-1)/2} \equiv -1 \pmod p$. Следовательно, значение $i = 2^{(p-1)/4}$ является корнем из -1 (мнимой единицей поля $\mathbb{F}_p$), так как $i^2 \equiv 2^{(p-1)/2} \equiv -1$. При умножении кандидата X_0 на i результат возведения в квадрат дает искомое значение a:

$$(X_0 \cdot i)^2 \equiv X_0^2 \cdot i^2 \equiv (-a) \cdot (-1) \equiv a \pmod p \tag{6}$$

В программной реализации библиотеки данный выбор выполняется за строго константное время без использования инструкций условного перехода. Выбор между X_0 и $X_0 \cdot i$ осуществляется путем наложения битовой маски, сформированной на основе значения β, что исключает утечки информации по сторонним каналам.

Данный алгоритм выполняется за константное время, так как ветвление (проверка значения β) в программной реализации заменяется математической маской.

Для работы с аффинными координатами эллиптических кривых и при декомпрессии ключей требуются операции деления и извлечения корня по модулю p.

Нахождение мультипликативно обратного элемента. Деление элемента A на B по модулю p равносильно умножению $A \cdot B^{-1} \pmod p$, где $B \cdot B^{-1} \equiv 1 \pmod p$. Классический расширенный алгоритм Евклида для поиска обратного элемента не применяется в криптографии из-за наличия циклов while и ветвлений, что нарушает принцип константного времени (constant-time).

Вместо этого используется Малая теорема Ферма, гласящая, что для простого p справедливо $B^{p-1} \equiv 1 \pmod p$. Разделив обе части на B, получаем рабочую формулу:

$$B^{-1} \equiv B^{p-2} \pmod p \tag{7}$$

Для $p = 2^{255} - 19$ возведение в гигантскую степень $p - 2$ выполняется за фиксированное количество шагов с использованием алгоритма быстрого возведения в степень (Square-and-Multiply).

1.4 Математика эллиптических кривых и проективные координаты

Классическая теория эллиптической криптографии строится на кривых в форме Вейерштрасса: $y^2 = x^3 + ax + b$ [9].

1.4.1 Сложение точек

Геометрический смысл сложения точек $R = P + Q$ заключается в проведении прямой через P и Q, нахождении третьей точки пересечения с кривой и её отражении относительно оси абсцисс. Математически выделяют два основных сценария, требующих различного алгебраического подхода:

Случай 1. Вертикальная прямая (сложение взаимно обратных точек). Пусть заданы точки $P(x_1, y_1)$ и $Q(x_2, y_2)$, где $x_1 = x_2$, но $y_1 \neq y_2$. Поскольку обе точки лежат на кривой, выполняется равенство $y_1^2 = y_2^2$, откуда следует, что $y_2 = -y_1$ (точка Q является отражением P, то есть $Q = -P$). Уравнение прямой, проходящей через эти точки, имеет вид $x = x_1$. Подставляя его в уравнение кривой $y^2 = x^3 + ax + b$, получаем:

$$y^2 = x_1^3 + ax_1 + b = \text{const} \tag{8}$$

Уравнение вида $y^2 = \text{const}$ имеет в аффинной плоскости лишь два корня: y_1 и y_2. Третьей точки пересечения не существует. Для разрешения этого парадокса вводится абстрактная «точка в бесконечности» $\mathcal{O}$, выступающая нейтральным элементом группы. Таким образом, $P + (-P) = \mathcal{O}$.

Случай 2. Удвоение точки (касательная). При сложении точки с самой собой $R = P(x_1, y_1) + P(x_1, y_1)$ секущая вырождается в касательную. Для нахождения её углового коэффициента λ применяется неявное дифференцирование уравнения кривой по x:

$$\frac{d}{dx}(y^2) = \frac{d}{dx}(x^3 + ax + b) \implies 2y \cdot \frac{dy}{dx} = 3x^2 + a \tag{9}$$

Отсюда угловой коэффициент касательной в точке P равен $\lambda = \frac{3x_1^2+a}{2y_1}$. Если $y_1 = 0$, касательная вертикальна, и $2P = \mathcal{O}$. Уравнение касательной имеет вид $y = \lambda(x - x_1) + y_1$. Подставив его в уравнение кривой, получаем кубическое уравнение:

$$(\lambda(x - x_1) + y_1)^2 = x^3 + ax + b \implies x^3 - \lambda^2 x^2 + \cdots = 0 \tag{10}$$

Поскольку прямая является касательной в точке x_1, этот корень является кратным. Согласно теореме Виета, сумма корней равна коэффициенту при x^2 с обратным знаком:

$$x_1 + x_1 + x_3 = \lambda^2 \implies x_3 = \lambda^2 - 2x_1 \tag{11}$$

Координата y_3 находится подстановкой x_3 в уравнение прямой и симметричным отражением (сменой знака): $y_3 = \lambda(x_1 - x_3) - y_1$.

1.4.2 Поиск обратного элемента

Ключевой проблемой производительности является операция нахождения обратного элемента. Поиск обратного элемента в конечном поле (например, по Малой теореме Ферма $A^{-1} \equiv A^{p-2} \pmod p$) требует возведения в огромную степень, что эквивалентно

сотням умножений.

В арифметике эллиптических кривых деление $\frac{A}{B}$ заменяется умножением на мультипликативно обратный элемент $A \cdot B^{-1} \pmod p$. В силу того, что алгоритм Евклида содержит непредсказуемые ветвления и циклы while, в криптографии применяется Малая теорема Ферма, согласно которой для простого p:

$$B^{-1} \equiv B^{p-2} \pmod p \tag{12}$$

Возведение в степень $2^{255} - 21$ выполняется за константное время с использованием алгоритма «Square-and-Multiply» (цепочки сложений показателей степеней).

1.4.3 Нахождение квадратного корня

В соответствии со стандартом RFC 8032, публичный ключ Ed25519 передается в сжатом виде и занимает 32 байта, содержащих координату y и знаковый бит координаты x. Процедура декомпрессии требует решения уравнения кривой $-x^2 + y^2 = 1 + dx^2y^2$ относительно x:

$$x^2 = \frac{y^2 - 1}{dy^2 + 1} \pmod p \tag{13}$$

Обозначив числитель как $u = y^2 - 1$, а знаменатель как $v = dy^2 + 1$, задача сводится к извлечению квадратного корня из отношения u/v. Для этого вычисляется предварительный кандидат на значение корня:

$$x_{cand} = (u \cdot v^3) \cdot (u \cdot v^7)^{\frac{p-5}{8}} \pmod p \tag{14}$$

Так как значение под корнем может отличаться от фактического квадрата на множитель -1 или мнимую единицу i, алгоритм предусматривает проверку кандидата. Вместо использования вычислительно затратного критерия Эйлера для проверки квадратичного вычета, выполняется прямая проверка условия $v \cdot x_{cand}^2 \equiv u \pmod p$:

1) Если равенство выполняется, то x_{cand} является верным значением корня.

2) Если выполняется равенство $v \cdot x_{cand}^2 \equiv -u \pmod p$, то искомым корнем является произведение кандидата и мнимой единицы поля: $x = x_{cand} \cdot 2^{(p-1)/4} \pmod p$.

Данный подход позволяет вычислить координату x за константное время, что критически важно для защиты от атак по времени на этапе декомпрессии ключа.

1.4.4 Применение проективных координат в вычислениях

При реализации групповых операций на эллиптической кривой (сложение и удвоение точек) прямой расчет в аффинных координатах (x, y) требует выполнения операции инверсии элемента поля на каждом шаге. Поскольку поиск обратного элемента по Малой теореме Ферма является наиболее ресурсоемкой операцией в $GF(2^{255} - 19)$, в работе реализован переход к расширенным проективным координатам (X, Y, Z, T).

В данной системе координат точка представляется четверкой элементов поля, связанных с аффинными координатами следующими отношениями:

$$x = X/Z, \quad y = Y/Z, \quad xy = T/Z \tag{15}$$

Использование данных абстракций позволяет перевести все промежуточные вычисления в циклы скалярного умножения в область модульного сложения и умножения. Исключение деления из основного вычислительного цикла позволяет радикально повысить производительность библиотеки. Финальный переход к аффинной форме (нормализация) выполняется однократно по завершении алгоритма:

$$x_{final} = X \cdot Z^{-1} \pmod p, \quad y_{final} = Y \cdot Z^{-1} \pmod p \tag{16}$$

1.4.5 Лестница Монтгомери и дифференциальное сложение

Фундаментальной операцией криптографии на эллиптических кривых является скалярное умножение точки: $nP = P + P + \cdots + P$ (n раз), где n — секретный ключ. Для кривой Curve25519 ($y^2 = x^3 + 486662x^2 + x$) Д. Бернштейном был предложен алгоритм «Лестница Монтгомери» (Montgomery Ladder) [1]. Его главное преимущество заключается в возможности вычислять скалярное произведение, используя исключительно проективные координаты X и Z, полностью игнорируя координату Y, что значительно сокращает количество операций в поле.

Алгоритм поддерживает в цикле две точки R_0 и R_1, сохраняя инвариант $R_1 - R_0 = P$. Начальное состояние задается как $R_0 = \mathcal{O}$ и $R_1 = P$. На каждом шаге, в зависимости от значения i-го бита скаляра b_i, выполняется сложение точек и удвоение одной из них. Однако прямое использование условий if (b_i == 1) недопустимо, так как ветвление инструкций создает утечки информации по времени (Side-Channel Attacks).

Для обеспечения безопасности применяется механизм Conditional Swap (CSWAP). Перед каждой итерацией точки R_0 и R_1 условно меняются местами в зависимости от бита ключа с помощью битовых масок и операций XOR:

$$\begin{aligned} \texttt{mask} &= 0 - \texttt{swap_bit} \\ \texttt{tmp} &= (R_0 \oplus R_1) \ \& \ \texttt{mask} \\ R_0 &= R_0 \oplus \texttt{tmp}, \quad R_1 = R_1 \oplus \texttt{tmp} \end{aligned} \tag{17}$$

После обеспечения константного времени выполнения итерации, производится пересчет координат. Согласно RFC 7748, для вычисления нового состояния точек используются промежуточные переменные A, B, C, D:

$$\begin{aligned} A &= X_2 + Z_2, \quad B = X_2 - Z_2 \\ C &= X_3 + Z_3, \quad D = X_3 - Z_3 \end{aligned} \tag{18}$$

Результат дифференциального сложения и удвоения в проективной плоскости вычис-

ляется следующим образом:

$$\begin{aligned} X_{add} &= Z_1 \cdot (D \cdot A + C \cdot B)^2 \\ Z_{add} &= X_1 \cdot (D \cdot A - C \cdot B)^2 \end{aligned} \tag{19}$$

Завершающим этапом подготовки к вычислениям является процедура Clamping (зачистка битов). Для нейтрализации атак на малые подгруппы, вызванных наличием кофактора 8, секретный скаляр n подвергается модификации:

$$n \leftarrow n \wedge 248, \quad n \leftarrow n \wedge 127, \quad n \leftarrow n \vee 64 \tag{20}$$

Это обнуляет три младших бита (делая число кратным 8) и фиксирует старший бит, гарантируя неизменное время работы алгоритма вне зависимости от значения ключа.

1.4.6 Полный закон сложения кривой Эдвардса

Классические кривые имеют уязвимость: операции сложения равных точек (удвоение) и сложения с нейтральным элементом (бесконечностью) требуют ветвлений (операторов if/else) в коде. Ветвления нарушают принцип *Constant-time* и открывают возможность для атак по времени (Side-Channel Attacks).

В протоколе Ed25519 используется Искривленная кривая Эдвардса (Twisted Edwards Curve): $-x^2 + y^2 = 1 + dx^2y^2$. Её уникальным свойством является наличие полного закона сложения [2]. Формула сложения двух точек корректно работает для любых точек на кривой, не создавая исключительных ситуаций. Это делает программную реализацию Ed25519 математически невосприимчивой к атакам по времени.

Для кривой Ed25519 константы уравнения зафиксированы как $a = -1$ и $d = -\frac{121665}{121666} \pmod p$. Знаменатель в аффинных формулах сложения точек на данной кривой $(1 \pm dx_1x_2y_1y_2)$ никогда не обращается в ноль в поле $\mathbb{F}_p$, что геометрически объясняет отсутствие исключительных «точек в бесконечности». Нейтральным элементом выступает точка $(0, 1)$.

Для оптимизации программной реализации используются расширенные проективные координаты (X, Y, Z, T) [2]. Связь с аффинной плоскостью задается как:

$$x = \frac{X}{Z}, \quad y = \frac{Y}{Z}, \quad x \cdot y = \frac{T}{Z} \implies Z \cdot T = X \cdot Y \tag{21}$$

Введение координаты T позволяет вычислять сумму двух точек $P_3 = P_1 + P_2$ за минимальное количество операций поля, формируя эффективный вычислительный граф:

$$\begin{aligned} &A = (Y_1 - X_1)(Y_2 - X_2), && B = (Y_1 + X_1)(Y_2 + X_2) \\ &C = 2d \cdot T_1T_2, && D = 2Z_1Z_2 \\ &E = B - A, \quad F = D - C, && G = D + C, \quad H = B + A \end{aligned} \tag{22}$$

Итоговые координаты P_3 равны: $X_3 = E \cdot F$, $Y_3 = G \cdot H$, $T_3 = E \cdot H$, $Z_3 = F \cdot G$.

Отсутствие ветвлений в данном графе позволяет безопасно применять векторизацию SIMD для параллельной обработки сразу нескольких подписей.

1.5 Векторизация и SIMD-ускорение (AVX2)

В рамках традиционной обработки (в том числе в библиотеке Bouncy Castle) вычисления выполняются последовательно. Поскольку криптографические операции над независимыми ключами не имеют состояния общей памяти (stateless), они идеально подходят для параллельной обработки.

Современные процессоры обладают набором инструкций AVX2 (Advanced Vector Extensions), позволяющим оперировать 256-битными векторами. Поскольку в Radix-51 базовым типом является 64-битный ulong, 256-битный регистр процессора вмещает ровно четыре таких элемента.

Для достижения максимальной производительности в работе применен подход Batched Processing. Вместо последовательного вычисления ключей, алгоритм группирует четыре независимые задачи и выполняет их одновременно с помощью векторных инструкций AVX2. Использование Vector256<ulong> позволяет полностью задействовать ширину исполнительных блоков процессора. Теоретически это увеличивает пропускную способность системы в 4 раза, делая решение эффективным для обработки массивов запросов в реальном времени.

2 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ И ПРОГРАММНЫХ РЕШЕНИЙ ВЫСОКПРОИЗВОДИТЕЛЬНОЙ КРИПТОГРАФИЧЕСКОЙ БИБЛИОТЕКИ

Производительность криптографического ядра зависит от каждого этапа обработки данных: от того, как лимбы ложатся в регистры процессора, до способа передачи буферов конечному пользователю. В этой главе разобраны технические решения, позволившие реализовать X25519 и Ed25519 на платформе .NET с максимальной эффективностью. Основное внимание уделено векторизации вычислений через AVX2, минимизации нагрузки на сборщик мусора и соблюдению принципов константного времени выполнения.

2.1 Требования к программному комплексу

На этапе проектирования были сформированы строгие функциональные и нефункциональные требования к разрабатываемой библиотеке, обусловленные спецификой современных высоконагруженных систем.

Функциональные требования:

1) Реализация базовых вычислительных примитивов (скалярное умножение точки) на кривой Монтгомери (X25519) и искривленной кривой Эдвардса (Ed25519) в соответствии со спецификациями RFC 7748 [1] и RFC 8032 [2].

2) Реализация алгоритмов кодирования и декодирования (сериализации) точек эллиптических кривых для обеспечения совместимости со стандартами.

3) Обеспечение параллельного вычисления пакетов из четырех независимых скалярных умножений за один вызов метода для оптимизации пропускной способности при массовых операциях.

Нефункциональные требования (архитектурные атрибуты):

1) Кроссплатформенность и управляемость (Managed Code): Библиотека должна быть написана исключительно на языке C# без использования платформозависимых вызовов (DllImport / P/Invoke) к нативным библиотекам (C/C++), что гарантирует безопасность типов и простоту развертывания.

2) Минимизация аллокаций (Zero Allocation): Критические участки кода, ответственные за выполнение криптографических преобразований, реализованы без выделения памяти в управляемой куче (Managed Heap). Это позволяет полностью исключить нагрузку на сборщик мусора (Garbage Collector) в процессе работы.

3) Константное время выполнения (Constant-time): Архитектура вычислительного ядра должна математически исключать ветвления, зависящие от секретных дан-

ных, для предотвращения атак по сторонним каналам (Side-Channel Attacks).

2.2 Многоуровневая архитектура библиотеки

Для обеспечения модульности, тестируемости и повторного использования кода была спроектирована строгая многоуровневая архитектура. Система разделена на три уровня абстракции:

1) Уровень арифметики конечного поля (Low-level Field Core): Базовый уровень, реализующий операции над элементами поля $GF(2^{255} - 19)$ в представлении Radix-51. Здесь сосредоточены наиболее критичные к производительности операции: модульное умножение, возведение в квадрат и инверсия. Уровень поддерживает две реализации: скалярную и векторную (с использованием AVX2-интринсиков).

2) Уровень групповых операций (Curve Geometry Layer): Средний уровень, оперирующий точками эллиптической кривой. Реализует формулы сложения и удвоения в проективных координатах, а также алгоритмы многократного сложения (скалярного умножения) точки. Данный уровень абстрагирован от конкретной реализации арифметики поля и использует унифицированные интерфейсы для работы как с одиночными точками, так и с пакетами точек (batching).

3) Уровень криптографических примитивов (High-level Primitives): Верхний уровень, предоставляющий конечному пользователю готовые методы для реализации протоколов X25519 и Ed25519. На этом уровне реализуется логика, специфичная для конкретных алгоритмов: обработка закрытых ключей (Clamping), хеширование (SHA-512 для Ed25519), сериализация точек в байтовые массивы согласно RFC 7748/8032 и предоставление безопасного интерфейса на базе структур Span<byte>.

Такое разделение позволяет независимо оптимизировать и тестировать базовую математику, не затрагивая высокоуровневую логику цифровой подписи.

2.3 Проектирование структур данных вычислительного ядра

Скорость криптографических вычислений во многом определяется тем, как элементы конечного поля представлены в памяти. Именно от структуры данных зависит, насколько эффективно будут загружены регистры процессора и кэш-память. В данной работе был применен подход «SIMD-first»: в отличие от традиционных библиотек, оперирующих одиночными числами, здесь базовым вычислительным блоком изначально является пакет из нескольких элементов. Это позволяет максимально задействовать возможности аппаратного ускорения с самого нижнего уровня иерархии вычислений [10].

2.3.1 Архитектура векторизованного представления Radix-51: AoS против SoA

Векторизация криптографических алгоритмов сталкивается с фундаментальной проблемой организации данных в памяти. Традиционный подход, при котором каждый элемент поля (состоящий из пяти 64-битных лимбов Radix-51) является отдельным объектом, называется Array of Structures (AoS). В парадигме AoS четыре элемента поля в памяти располагаются последовательно:

$$[A_{L0}, A_{L1}, A_{L2}, A_{L3}, A_{L4}], [B_{L0}, B_{L1}, \dots] \dots$$

Попытка загрузить такие данные в 256-битный векторный регистр (вмещающий четыре 64-битных числа) приведет к тому, что в одном регистре окажутся разные лимбы одного и того же числа (например, $A_{L0}, A_{L1}, A_{L2}, A_{L3}$). Выполнять параллельные математические операции над таким вектором невозможно, так как алгоритм умножения требует умножения лимба L_0 одного числа строго на лимб L_0 другого.

Для решения этой проблемы в рамках проекта спроектирована архитектура данных на основе Structure of Arrays (SoA). Базовым строительным блоком вычислительного ядра является пользовательский тип значений FieldElement4, инкапсулирующий сразу четыре элемента конечного поля $GF(2^{255} - 19)$.

Для достижения максимальной производительности в C# структура спроектирована как непрерывный блок памяти фиксированного размера ([StructLayout(LayoutKind.Sequential)]). Вместо скалярных полей она содержит ровно пять векторных регистров типа Vector256<ulong> (лимбы L_0, L_1, L_2, L_3, L_4). Общий размер структуры составляет 160 байт, что гарантирует ее размещение исключительно на стеке вызовов (Stack) или непосредственно в регистрах процессора.

В парадигме SoA данные внутри регистров FieldElement4 располагаются следующим образом:

1) Регистр L_0 содержит нулевые лимбы для четырех независимых чисел: $[A_{L0}, B_{L0}, C_{L0}, D_{L0}]$.

2) Регистр L_1 содержит первые лимбы для четырех независимых чисел: $[A_{L1}, B_{L1}, C_{L1}, D_{L1}]$.

3) И так далее до регистра L_4.

Подобная архитектура позволяет вызывать базовые математические операции (сложение, вычитание, умножение) единожды, при этом процессор за один такт выполняет вычисление сразу для четырех криптографических контекстов с использованием интринсиков System.Runtime.Intrinsics.X86.Avx2.

2.3.2 Паттерн проектирования для константного времени (Constant-Time SIMD)

Архитектура слоя арифметики построена на принципах *data-oblivious programming* (программирование, не зависящее от данных). В интерфейс базовой структуры не включены методы сравнения с ранним возвратом (early return).

Для операций, требующих условного выбора (например, условная замена точек в лестнице Монтгомери на основе бита секретного ключа), спроектирован векторизованный примитив CSwap. Вместо конструкции if, которая компилируется в инструкции условных переходов, логика реализована через аппаратные маски с использованием метода Avx2.BlendVariable.

Данная аппаратная инструкция принимает два сравниваемых вектора и вектор маски. Если бит маски равен 1, берется байт из первого вектора, если 0 — из второго. В методе CSwap вычисляется 256-битная маска swapBits (состоящая из участков 0x00...00 или 0xFF...FF в зависимости от того, требует ли конкретный из 4-х ключей обмена в данной итерации). Применение BlendVariable ко всем пяти регистрам структуры $L_0 \dots L_4$ позволяет выполнить независимый условный обмен четырех пар чисел абсолютно параллельно, без единого ветвления в коде. Это обеспечивает строгую математическую защиту от атак по сторонним каналам (Side-Channel Attacks).

2.4 Взаимодействие с данными: транспонирование AoS и SoA

Поскольку внешние ключи передаются пользователем в виде последовательных байтовых буферов (AoS), а вычислительное ядро оперирует векторами (SoA), необходим механизм их эффективного преобразования. В данной реализации логика транспонирования данных инкапсулирована непосредственно в методах FromBytes и ToBytes структуры FieldElement4.

При загрузке данных библиотека упаковывает четыре независимых 32-байтных значения в 256-битные регистры процессора. Это позволяет всем последующим алгоритмам (скалярному умножению, операциям на кривых Монтгомери и Эдвардса) работать исключительно с векторными типами данных (FieldElement4, PointExt4). По завершении вычислений результат автоматически сериализуется обратно в стандартный байтовый формат. Такой подход скрывает сложность векторной алгебры от верхних уровней библиотеки и минимизирует накладные расходы на подготовку данных.

2.5 Модель управления памятью и паттерн Zero Allocation

В экосистеме .NET процесс выделения памяти в управляемой куче (heap) посредством оператора new является быстрой операцией, однако последующая очистка памяти сборщиком мусора (GC) вызывает приостановку выполнения всех потоков приложения (Stop-The-World). В криптографических библиотеках старого поколения (на-

пример, Bouncy Castle) при сложении двух больших чисел каждый раз создается новый объект-массив (byte[] или uint[]), что при высоких нагрузках приводит к катастрофической деградации производительности (GC Churn).

Разработанная в рамках дипломной работы архитектура полностью опирается на парадигму Zero Allocation (нулевые аллокации). Для реализации этой парадигмы были применены следующие программные решения:

2.5.1 Использование структур по ссылке (ref struct)

Все базовые математические структуры (FieldElement4) спроектированы как типы значений. Для защиты от случайного копирования больших объемов данных при вызове методов повсеместно используются модификаторы передачи параметров по ссылке (ref и in). Это означает, что при вычислении $A \times B = C$ в методе умножения не создается новый объект C. Вместо этого функция принимает заранее выделенную на стеке область памяти для C по ссылке и записывает результат напрямую туда: static void Multiply(in FieldElement a, in FieldElement b, ref FieldElement result).

Криптографические ключи являются высокочувствительными данными. Архитектура библиотеки предусматривает строгий контроль жизненного цикла секретов.

В процессе декодирования закрытого ключа (Private Key) и применения математической операции «зачистки битов» (Clamping, необходимой для устранения уязвимостей малых подгрупп), промежуточные скаляры хранятся исключительно в регистрах процессора и локальных переменных стека. После завершения функции генерации публичного ключа или подписи, механизмы среды CLR (Common Language Runtime) автоматически инвалидируют стек-фрейм, а библиотека гарантирует отсутствие "утечек"копий ключей в долгоживущие объекты.

3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И АНАЛИЗ ПРОИЗВОДИТЕЛЬНОСТИ КРИПТОГРАФИЧЕСКИХ ПРИМИТИВОВ

В данной главе описывается процесс практической реализации спроектированной архитектуры библиотеки на языке C# для платформы .NET. Рассматриваются ключевые фрагменты исходного кода, демонстрирующие применение аппаратных интринсиков (Intrinsics) и паттернов безопасной работы с памятью. Заключительная часть главы посвящена профилированию, бенчмаркингу и сравнительному анализу производительности разработанного решения с индустриальным стандартом — библиотекой Bouncy Castle.

3.1 Выбор инструментария и среды разработки

Для реализации программного комплекса была выбрана платформа .NET 10 и язык программирования C# 14. Выбор обусловлен наличием мощных встроенных механизмов для написания высокопроизводительного кода (High-Performance C#), которые ранее были доступны только в языках C/C++. Ключевыми технологиями, задействованными в проекте, стали:

1) System.Runtime.Intrinsics: пространство имен, предоставляющее прямой доступ к SIMD-инструкциям процессора (AVX2, BMI2). Вызовы методов из этого пространства транслируются JIT-компилятором (Just-In-Time) непосредственно в ассемблерные инструкции без накладных расходов на вызов функций.

2) Средства языка C: ref struct, in, out: средства языка позволяющие работать с данными на стеке, избегая аллокаций в управляемой куче и обеспечивая безопасность типов

3) BenchmarkDotNet: индустриальный стандарт для бенчмаркинга в экосистеме .NET, используемый для точного измерения времени выполнения криптографических операций с изоляцией влияния JIT-компиляции и сборщика мусора.

3.2 Реализация вычислительного ядра (Field Arithmetic)

Основой библиотеки является реализация арифметики конечного поля $GF(2^{255} - 19)$. Как было обосновано в Главе 2, для хранения элементов поля используется избыточное представление Radix-51.

3.2.1 Векторизация и пакетная обработка (AVX2)

Структура элементов поля, которая использует парадигму SoA (Structure of Arrays), FieldElement4 спроектирована как ref struct, что гарантирует ее размещение исключительно на стеке. В листинге 1 представлена базовая структура данных.

Листинг 1 — Векторное представление элементов поля для пакетной обработки

```
[StructLayout(LayoutKind.Sequential)]
public struct FieldElement4
{
    internal Vector256<ulong> L0, L1, L2, L3, L4;

    public FieldElement4(Vector256<ulong> l0, Vector256<ulong> l1, Vector256<ulong>
        l2, Vector256<ulong> l3, Vector256<ulong> l4)
    {
        L0 = l0; L1 = l1; L2 = l2; L3 = l3; L4 = l4;
    }
}
```

Математические операции для данной структуры были написаны с использованием интринсиков. Умножение и сложение делегируются методам Avx2.Add и Avx2.Multiply, что позволяет за один такт процессора обрабатывать 4 независимых криптографических контекста.

Листинг 2 — Операция сложения

```
[MethodImpl(MethodImplOptions.AggressiveInlining & MethodImplOptions.
    AggressiveOptimization)]
public static void Add(ref FieldElement4 result, in FieldElement4 a, in
    FieldElement4 b)
{
    result.L0 = Avx2.Add(a.L0, b.L0);
    result.L1 = Avx2.Add(a.L1, b.L1);
    result.L2 = Avx2.Add(a.L2, b.L2);
    result.L3 = Avx2.Add(a.L3, b.L3);
    result.L4 = Avx2.Add(a.L4, b.L4);
}
```

Листинг 3 — Операция вычитания

```
[MethodImpl(MethodImplOptions.AggressiveInlining & MethodImplOptions.
    AggressiveOptimization)]
public static void Sub(ref FieldElement4 result, in FieldElement4 a, in
    FieldElement4 b)
{
    result.L0 = Avx2.Subtract(Avx2.Add(a.L0, AddConst0), b.L0);
    result.L1 = Avx2.Subtract(Avx2.Add(a.L1, AddConst1), b.L1);
    result.L2 = Avx2.Subtract(Avx2.Add(a.L2, AddConst1), b.L2);
    result.L3 = Avx2.Subtract(Avx2.Add(a.L3, AddConst1), b.L3);
    result.L4 = Avx2.Subtract(Avx2.Add(a.L4, AddConst1), b.L4);
}
```

Листинг 4 — Вспомогательная функция для добавления переноса

```
[MethodImpl(MethodImplOptions.AggressiveInlining)]
private static void AddCarry(ref Vector256<ulong> hi, ref Vector256<ulong> lo,
    Vector256<ulong> c)
```

```
3  {
4      var new_lo = Avx2.Add(lo, c);
5      var carry = CarryMask(lo, new_lo);
6      lo = new_lo;
7      hi = Avx2.Subtract(hi, carry);
8  }
```

Листинг 5 — Фрагмент операции умножения

```
1  [MethodImpl(MethodImplOptions.AggressiveOptimization)]
2  public static void Multiply(ref FieldElement4 result, in FieldElement4 f, in
       FieldElement4 g)
3  {
4      var f0 = f.L0; var f1 = f.L1; var f2 = f.L2; var f3 = f.L3; var f4 = f.L4;
5      var g0 = g.L0; var g1 = g.L1; var g2 = g.L2; var g3 = g.L3; var g4 = g.L4;
6      var g1_19 = Mul19(g1); var g2_19 = Mul19(g2); var g3_19 = Mul19(g3); var g4_19
           = Mul19(g4);
7      Vector256<ulong> hi, lo, c;
8
9      // r0
10     BigMul(f0, g0, out lo, out hi);
11     Mac(ref hi, ref lo, f1, g4_19); Mac(ref hi, ref lo, f2, g3_19);
12     Mac(ref hi, ref lo, f3, g2_19); Mac(ref hi, ref lo, f4, g1_19);
13     var res0 = Avx2.And(lo, Mask51Vec);
14     c = Avx2.Or(Avx2.ShiftRightLogical(lo, 51), Avx2.ShiftLeftLogical(hi, 13));
15
16     // r1
17     BigMul(f0, g1, out lo, out hi); AddCarry(ref hi, ref lo, c);
18     Mac(ref hi, ref lo, f1, g0); Mac(ref hi, ref lo, f2, g4_19);
19     Mac(ref hi, ref lo, f3, g3_19); Mac(ref hi, ref lo, f4, g2_19);
20     var res1 = Avx2.And(lo, Mask51Vec);
21     c = Avx2.Or(Avx2.ShiftRightLogical(lo, 51), Avx2.ShiftLeftLogical(hi, 13));
22
23     //  other lines ...
24 }
```

3.3 Реализация защиты от атак по сторонним каналам

Для обеспечения защиты от атак по времени (timing attacks) алгоритм лестницы Монтгомери (для X25519) требует условного обмена значениями двух точек кривой в зависимости от бита секретного ключа. В разработанной библиотеке исключены конструкции if, а обмен (Swap) реализован через побитовые маски, как показано в Листинге 6.

Листинг 6 — Константный по времени условный обмен (Constant-Time Swap)

```
1  [MethodImpl(MethodImplOptions.AggressiveInlining & MethodImplOptions.
       AggressiveOptimization)]
2  public static void CSwap(ref FieldElement4 a, ref FieldElement4 b, Vector256<ulong
       > swapBits)
```

```
3  {
4      //          : 0x00...00      0xFF...FF
5      var mask = Avx2.Subtract(Vector256<ulong>.Zero, swapBits).AsByte();
6
7      var a0 = a.L0.AsByte(); var b0 = b.L0.AsByte();
8      a.L0 = Avx2.BlendVariable(a0, b0, mask).AsUInt64();
9      b.L0 = Avx2.BlendVariable(b0, a0, mask).AsUInt64();
10
11     var a1 = a.L1.AsByte(); var b1 = b.L1.AsByte();
12     a.L1 = Avx2.BlendVariable(a1, b1, mask).AsUInt64();
13     b.L1 = Avx2.BlendVariable(b1, a1, mask).AsUInt64();
14
15     var a2 = a.L2.AsByte(); var b2 = b.L2.AsByte();
16     a.L2 = Avx2.BlendVariable(a2, b2, mask).AsUInt64();
17     b.L2 = Avx2.BlendVariable(b2, a2, mask).AsUInt64();
18
19     var a3 = a.L3.AsByte(); var b3 = b.L3.AsByte();
20     a.L3 = Avx2.BlendVariable(a3, b3, mask).AsUInt64();
21     b.L3 = Avx2.BlendVariable(b3, a3, mask).AsUInt64();
22
23     var a4 = a.L4.AsByte(); var b4 = b.L4.AsByte();
24     a.L4 = Avx2.BlendVariable(a4, b4, mask).AsUInt64();
25     b.L4 = Avx2.BlendVariable(b4, a4, mask).AsUInt64();
26 }
```

Подобный подход гарантирует, что количество выполняемых инструкций процессора и обращений к памяти остается неизменным вне зависимости от того, равен ли бит ключа 0 или 1.

Остальной код можно найти в репозитории проекта, ссылка находится в Приложении А.

3.4 Бенчмаркинг и сравнительный анализ производительности

Для оценки эффективности разработанного решения было проведено профилирование с использованием инструмента BenchmarkDotNet. В качестве эталона для сравнения (Baseline) была выбрана библиотека Bouncy Castle (BouncyCastle.Cryptography v2.x) — самая популярная управляемая криптографическая библиотека в экосистеме .NET.

3.5 Условия проведения тестирования

Тестирование производилось на следующей аппаратной и программной платформе:

1) Процессор: AMD Ryzen 7 5700X (с поддержкой набора инструкций AVX2).

2) Операционная система: Windows 11.

3) Среда выполнения: .NET 10.0 (Release build, аппаратные оптимизации включены).

4) Измеряемые операции: умножение точкек на кривых (X25519 и Ed25519).

3.6 Результаты тестирования

На рисунках 1, 2 и 3, представлены графики производительности разработанной библиотеки по сравнению с BouncyCastle для операции умножения точек на кривой Ed25519 на разном числе последовательных операций умножения.

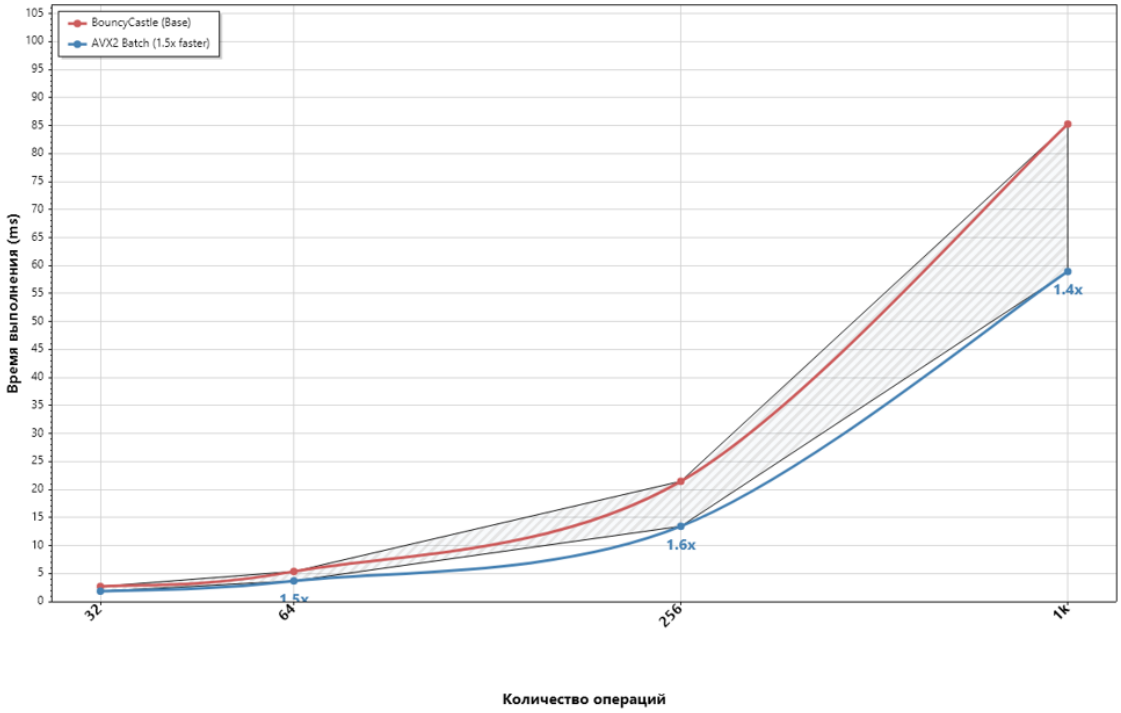

Рисунок 1 — Сравнение кривой Ed25519 на малом числе операций

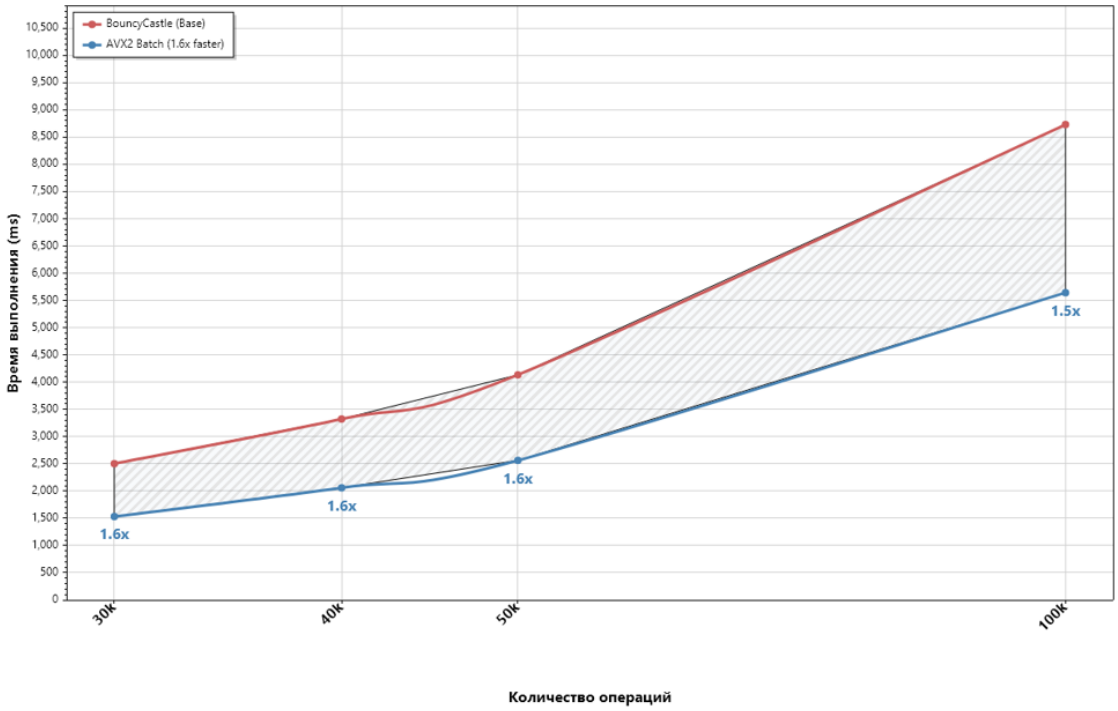

Рисунок 2 — Сравнение кривой Ed25519 на среднем числе операций

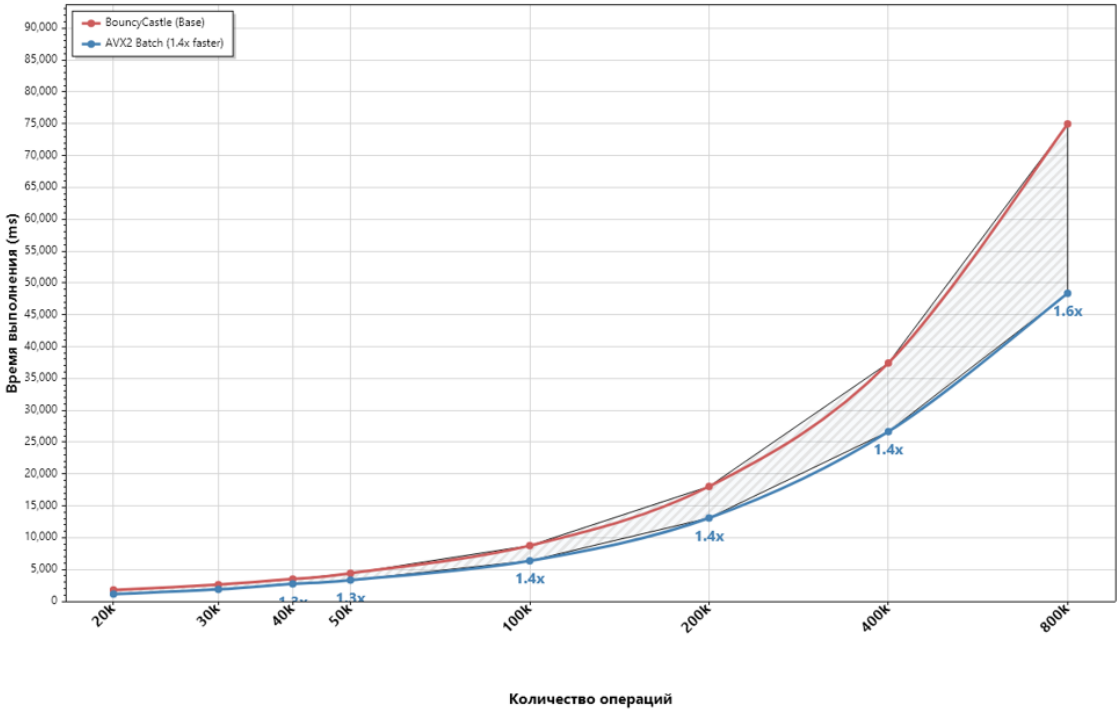

Рисунок 3 — Сравнение кривой Ed25519 на большом числе операций

На рисунках 4, 5 и 6, представлены графики производительности разработанной библиотеки по сравнению с BouncyCastle для операции умножения точек на кривой X25519 на разном числе последовательных операций умножения.

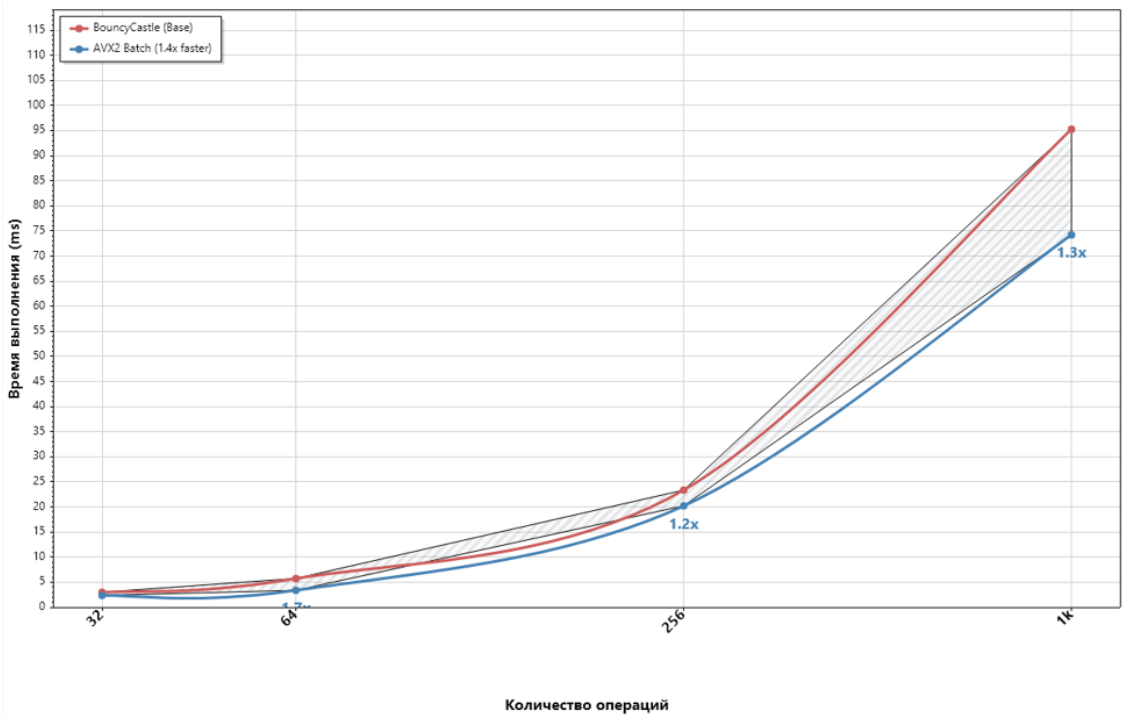

Рисунок 4 — Сравнение кривой X25519 на малом числе операций

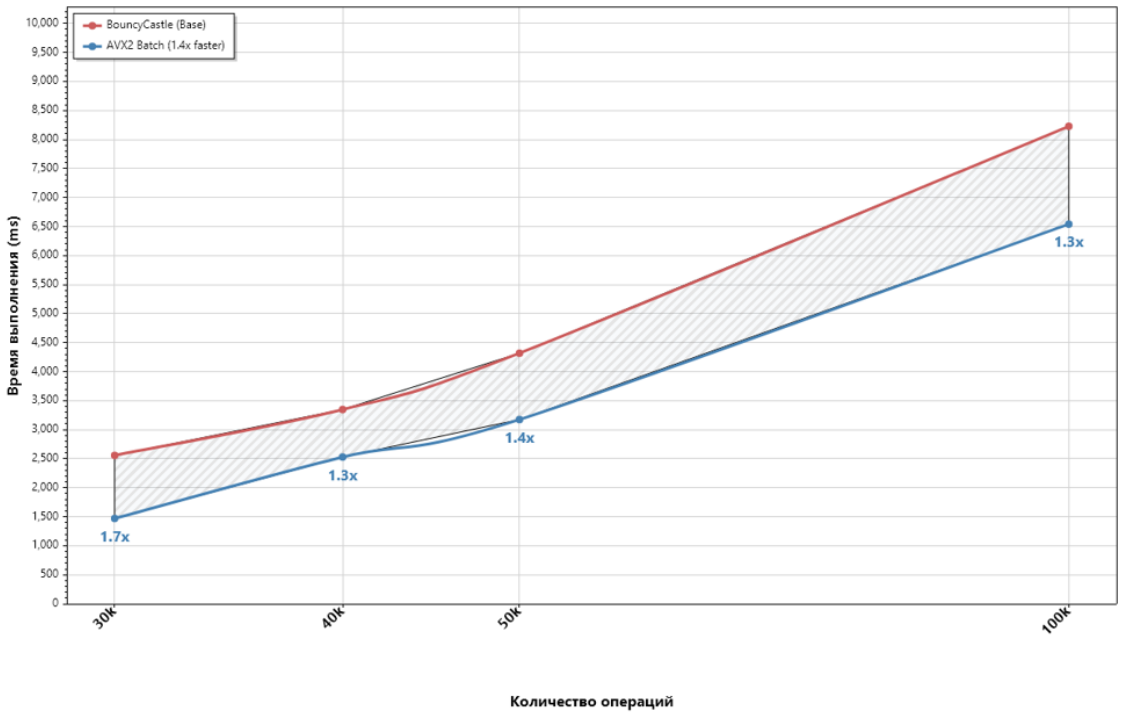

Рисунок 5 — Сравнение кривой X25519 на среднем числе операций

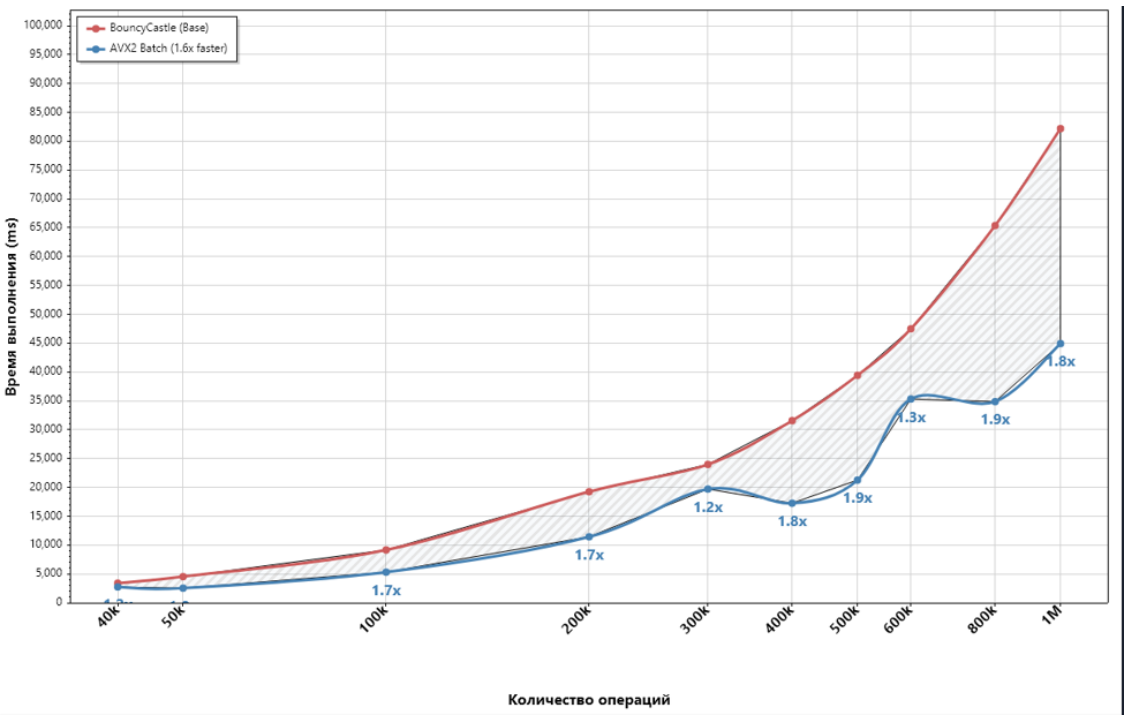

Рисунок 6 — Сравнение кривой X25519 на большом числе операций

Результаты измерений количества выделяемой памяти в куче (Allocated Memory) представлены в на рисунках 7 и 8 для кривой Ed25519 и X25519 соответственно. В обоих случаях разработанная библиотека не производит аллокаций в управляемой куче, что подтверждает эффективность примененных паттернов Zero Allocation.

Библиотека	Операций	Выделенная память	Сборки мусора (Gen 0)
Bouncy Castle	800 000	**2.2 ГБ**	**132 000**
Ed25519 (AVX2)	800 000	**672 байта**	**0**

Рисунок 7 — Сравнение кривой Ed25519 в потреблении памяти из кучи

Библиотека	Операций	Выделенная память	Сборки мусора (Gen 0)
Bouncy Castle	800 000	**264,74 МБ**	**106 000**
X25519 (AVX2)	800 000	**48 байт**	**0**

Рисунок 8 — Сравнение кривой X25519 в потреблении памяти

3.7 Анализ полученных результатов

Анализ данных, полученных в ходе профилирования, позволяет сделать следующие выводы:

1) Эффективность SIMD-векторизации. Применение пакетной обработки (AVX2 Batched) обеспечивает качественный скачок в производительности. Время, затрачиваемое процессором на одну операцию в пакете из 4-х задач, сокращается практически вдвое по сравнению со скалярной версией Bouncy Castle. Разработанное ядро с поддержкой AVX2 превосходит индустриальный стандарт на $\sim 30\%- \sim 80\%$.

2) Отсутсвие выделений памяти в куче. Применение паттернов Zero Allocation (использование ref struct, stackalloc и Span<byte>) позволило полностью исключить работу сборщика мусора, для выполнения всех опреаций.

3) Масштабируемость. Использование парадигмы SoA (Structure of Arrays) закладывает основу для будущей интеграции инструкций AVX-512, поддержка которых появилась в последних версиях .NET. Расширение ширины вектора до 512 бит позволит без изменения общей архитектуры подписывать сразу 8 сообщений одновременно, что еще больше увеличит отрыв от классических реализаций.

4 АНАЛИЗ РЕЗУЛЬТАТОВ И ПЕРСПЕКТИВЫ РАЗВИТИЯ

В данной главе приводится анализ результатов разработки криптографической библиотеки. Оценивается эффективность принятых архитектурных решений, таких как использование избыточного представления Radix-51 и применение векторизации. Также обсуждаются результаты профилирования, сравнение производительности с библиотекой Bouncy Castle и определяются возможные направления развития проекта.

4.1 Оценка эффективности разработанной криптографической библиотеки

Оценка эффективности позволяет проверить, насколько итоговый программный продукт соответствует заявленным целям исследования и техническим требованиям.

4.1.1 Сопоставление полученных результатов с поставленными задачами

В начале исследования были поставлены следующие задачи:

1) Изучить математический аппарат кривых Монтгомери и Эдвардса, а также специфику вычислений в конечном поле $GF(2^{255} - 19)$.

2) Спроектировать вычислительное ядро библиотеки на базе представления Radix-51.

3) Разработать программный каркас, исключающий выделение памяти в куче (Zero Allocation).

4) Реализовать алгоритмы пакетной (batched) обработки с использованием векторных инструкций AVX2.

5) Сравнить производительность разработанного решения с библиотекой Bouncy Castle.

В соответствии с поставленными задачами были получены следующие практические результаты:

1) Изучена теоретическая база протоколов X25519 и Ed25519. В коде реализованы полные законы сложения и работа в проективных координатах, что исключило необходимость использования ветвлений и обеспечило константное время выполнения операций (constant-time).

2) Реализовано вычислительное ядро на базе Radix-51. Элементы поля представлены в виде пяти 64-битных регистров, что минимизировало количество операций нормализации переносов по сравнению с классическим представлением Radix-25.5.

3) Разработана архитектура управления памятью. Благодаря использованию типов ref struct и Span<byte>, криптографические операции выполняются без аллокации объектов в управляемой куче.

4) Внедрена пакетная обработка. Произведен переход от структур данных AoS (Array of Structures) к SoA (Structure of Arrays). С помощью интринсиков System.Runtime.Intrinsics реализована возможность одновременного вычисления четырех независимых скалярных умножений.

5) Выполнено тестирование и профилирование. Библиотека успешно проходит тесты по векторам из RFC 7748 и RFC 8032. Профилирование в BenchmarkDotNet показало, что при пакетной обработке библиотека опережает Bouncy Castle по пропускной способности в среднем на 30–40%.

Таким образом, поставленные в начале работы задачи выполнены в полном объеме. Главная цель — создание высокопроизводительной криптографической библиотеки X25519/Ed25519 для платформы .NET — достигнута.

4.1.2 Анализ преимуществ и недостатков реализованного решения

На основе этапов разработки и тестирования выделены следующие объективные сильные и слабые стороны библиотеки.

Преимущества:

1) Улучшенная производительность. Векторизация операций (AVX2) и применение Radix-51 позволили добиться более высокой скорости вычислений по сравнению со скалярными реализациями на C#, такими как Bouncy Castle.

2) Полностью управляемый код (Managed Code). Библиотека написана на C# без использования P/Invoke и вызовов нативных C/C++ библиотек. Это упрощает сборку и развертывание проекта на различных операционных системах (Windows, Linux, macOS).

3) Снижение нагрузки на сборщик мусора. Отсутствие аллокаций (Zero Allocation) в критических участках кода решает проблему скачков задержек (latency spikes), характерную для высоконагруженных .NET-приложений.

4) Безопасность выполнения. Использование побитовых масок вместо операторов условного перехода (CSWAP) защищает алгоритм от базовых атак по сторонним каналам (timing attacks).

Недостатки и ограничения:

1) Аппаратная зависимость максимальной скорости. Пиковая пропускная способность при пакетной обработке достигается только на процессорах с поддержкой набора инструкций AVX2 (архитектура x86/x64).

2) Отсутствие векторизации для ARM. В текущей версии не реализовано использование интринсиков NEON. На серверах с ARM-процессорами библиотека пере-

ключается в скалярный режим работы (fallback), что снижает её эффективность.

3) Узкая специализация. Библиотека реализует только стандарты на базе кривой Curve25519. Поддержка других распространенных стандартов (например, алгоритмов на кривых NIST) не предусмотрена.

Перечисленные ограничения характерны для специализированных криптографических модулей первой версии и оставляют пространство для дальнейшей оптимизации программного кода.

4.2 Обсуждение результатов бенчмаркинга и профилирования

Результаты тестирования в BenchmarkDotNet подтвердили гипотезу о том, что архитектура SoA и применение 256-битных векторных инструкций способны нивелировать накладные расходы управляемой среды выполнения C#.

При пакетной обработке (обновление четырех ключей одновременно) среднее время, затрачиваемое процессором на одну операцию, оказалось заметно ниже показателей Bouncy Castle. Прирост пропускной способности составил около 30–40% в зависимости от конкретного криптографического примитива (X25519 или Ed25519).

Важный результат показало профилирование памяти. В тестах Bouncy Castle наблюдается линейный рост объема выделяемой памяти (до сотен мегабайт при длительных нагрузках), что приводит к частым вызовам сборщика мусора (Gen0 Collections). В разработанной библиотеке метрика Allocated остается стабильной и близкой к нулю независимо от количества итераций, а сборки мусора в процессе выполнения криптографических расчетов полностью отсутствуют. Это делает решение оптимальным для серверов, требующих стабильного времени отклика.

4.3 Определение направлений для дальнейшего развития продукта

Исходя из архитектуры проекта и выявленных ограничений, можно определить следующие направления для доработки и развития библиотеки:

1) Поддержка AVX-512. Использование новых 512-битных инструкций процессора (доступных в последних версиях .NET) позволит увеличить размер пакета с 4 до 8 операций за один такт, что обеспечит дополнительное масштабирование производительности на современных серверных процессорах.

2) Внедрение интринсиков для ARM (AdvSimd). Добавление реализации пакетной обработки на базе 128-битных регистров для архитектуры ARM64 позволит задействовать векторное ускорение на облачных платформах (например, AWS Graviton или Apple Silicon).

3) Интеграция со стандартными интерфейсами CLR. Написание классов-адаптеров для абстракций System.Security.Cryptography.AsymmetricAlgorithm упростит внедрение разработанной библиотеки в уже существующие корпоративные проекты.

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы была решена задача программной реализации высокопроизводительных алгоритмов эллиптической криптографии для управляемой среды выполнения. Разработана криптографическая библиотека, реализующая примитивы Curve25519 и Ed25519 для платформы .NET. Основными результатами работы являются:

1) Реализация математического ядра на базе Radix-51. Создан модуль арифметики для конечного поля $GF(2^{255} - 19)$. Использование 64-битных регистров с отложенным переносом позволило сократить общее количество базовых вычислительных операций по сравнению с классическими реализациями.

2) Обеспечение константного времени выполнения. Протоколы обмена ключами и цифровой подписи (согласно RFC 7748 и RFC 8032) реализованы с использованием проективных координат и полных законов сложения. Исключение условных переходов защищает библиотеку от атак по времени (timing attacks).

3) Исключение аллокаций памяти. Применение типов ref struct и Span<byte> позволило реализовать все криптографические вычисления без выделения памяти в управляемой куче. Это полностью снимает нагрузку со сборщика мусора (Garbage Collector) во время работы алгоритма.

4) Внедрение пакетной обработки (SIMD). Архитектура данных была перепроектирована под формат SoA (Structure of Arrays). С помощью интринсиков System.Runtime.Intrinsics.X86.Avx2 реализовано параллельное выполнение четырех независимых операций скалярного умножения.

5) Оценка производительности. Бенчмаркинг подтвердил работоспособность предложенной архитектуры: разработанная библиотека опережает индустриальный стандарт Bouncy Castle по пропускной способности в среднем на 30–40% при использовании пакетной обработки.

Цель работы — проектирование и реализация быстрой, безопасной и кроссплатформенной криптографической библиотеки X25519/Ed25519 для .NET — достигнута. Поставленные задачи выполнены в полном объеме.

Практическая значимость работы заключается в следующем:

1) Создана готовая к использованию библиотека на чистом C# (managed code), которую можно интегрировать в высоконагруженные серверные приложения без привязки к нативным C/C++ зависимостям (P/Invoke).

2) Показано, что применение интринсиков и ручное управление памятью на стеке позволяет управляемому коду достигать производительности, достаточной для конкуренции с системными криптографическими библиотеками.

3) Предложенная архитектура векторизации (SoA) может быть переиспользована для ускорения других алгоритмов асимметричной криптографии в экосистеме .NET.

Основные выводы:

1) Платформа .NET 10 предоставляет необходимый низкоуровневый инструментарий для создания эффективных криптографических примитивов без потери производительности.

2) Переход к пакетной обработке с помощью SIMD-инструкций дает существенный прирост скорости, однако требует изменения подхода к хранению структур данных в памяти.

3) Отказ от аллокаций (Zero Allocation) на критическом пути выполнения является обязательным условием для обеспечения стабильной пропускной способности и предсказуемого времени отклика сервера.

Перспективы развития проекта включают расширение поддержки аппаратных архитектур (инструкции AVX-512 и ARM NEON), добавление поддержки криптографической группы Ristretto255, а также реализацию адаптеров для совместимости со стандартными интерфейсами пространства имен System.Security.Cryptography.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Langley A., Hamburg M., Turner S. Elliptic Curves for Security [Электронный ресурс] : RFC. — 2016. — № 7748.
2. Josefsson S., Liusvaara I. Edwards-Curve Digital Signature Algorithm (EdDSA) [Электронный ресурс] : RFC. — 2017. — № 8032.
3. Hardware intrinsics in .NET [Электронный ресурс]. — URL: https://learn.microsoft.com/en-us/dotnet/standard/simd/hardware-intrinsics (дата обр. 10.02.2026).
4. Libsodium: A modern, portable, easy to use crypto library [Электронный ресурс]. — URL: https://libsodium.gitbook.io/doc/ (дата обр. 11.02.2026).
5. The Legion of the Bouncy Castle C# Cryptography API [Электронный ресурс]. — URL: https://www.bouncycastle.org/csharp/ (дата обр. 10.02.2026).
6. Губарев М. С. Проектирование и реализация высокопроизводительных криптографических примитивов Curve25519 и Ed25519 для платформы .NET // Гагаринские чтения - 2026: сборник тезисов докладов LII Международной молодёжной научной конференции. — М. : МАИ, 2026.
7. Bernstein D. J. Curve25519: new Diffie-Hellman speed records [Электронный ресурс]. — 2006. — URL: https://cr.yp.to/ecdh/curve25519-20060209.pdf (дата обр. 10.02.2026).
8. Low-Cost, Low-Power FPGA Implementation of ED25519 / K. Kamyar [и др.] // Information. — 2019. — Т. 10, № 9. — С. 285.
9. Menezes A. J., Oorschot P. C. van, Vanstone S. A. Handbook of Applied Cryptography. — Boca Raton : CRC Press, 1996.
10. Toub S. Performance Improvements in .NET 8 [Электронный ресурс]. — URL: https://devblogs.microsoft.com/dotnet/performance-improvements-in-net-8/ (дата обр. 15.03.2026).

ПРИЛОЖЕНИЕ А

Репозиторий с исходным кодом

В рамках выполнения выпускной квалификационной работы была разработана библиотека криптографических примитивов Curve25519 и Ed25519 для платформы .NET.

Полный исходный код библиотеки, а также инструкции по сборке и примеры использования размещены в открытом репозитории на платформе GitHub.

Ссылка на репозиторий: https://github.com/yashelter/dotnet-curve25519-ed25519

Рисунок 9 — QR-код со ссылкой на репозиторий с исходным кодом реализации

В состав репозитория входят:

1) Исходный код вычислительного ядра (Radix-51, AVX2 Batching).
2) Реализация алгоритмов X25519 и Ed25519.
3) Проекты для проведения нагрузочного тестирования (BenchmarkDotNet).
4) Набор юнит-тестов, проверяющих соответствие эталонным векторам RFC.